WIDS: Introduction to Reinforcement Learning

Winter in Data Science (WiDS 5.0) Final Report

Introduction

Reinforcement learning represents a paradigm shift in machine learning, enabling agents to learn optimal behaviors through interaction with their environment without explicit supervision. Unlike supervised learning, where the learner is provided with labeled examples, RL agents learn through trial and error, receiving only scalar reward signals that guide their learning trajectory. This report documents our comprehensive study of reinforcement learning theory and implementation, based on the foundational text by Sutton and Barto (2018), supplemented by contemporary applied RL literature.

The motivation for this study stems from RL's profound applications in autonomous systems, game playing, robotics, and sequential decision-making. Our objective was to understand core RL concepts from first principles, implement canonical algorithms, and apply them to increasingly complex problem domains. Through four weeks of intensive study and implementation, we progressed from simple bandit problems to solving the 15-puzzle using deep reinforcement learning principles.

The theoretical framework underlying all our work is the Markov Decision Process (MDP), which provides a mathematical abstraction for sequential decision-making under uncertainty. By understanding MDPs, value functions, policies, and dynamic programming methods, we developed the mathematical maturity to tackle both classical control problems and modern deep RL applications. This report synthesizes our learning journey, presenting both theoretical foundations and practical implementations with empirical validation.

Week 1: Value Functions, Policies, and the Multi-Armed Bandit Problem

Theoretical Foundations

In reinforcement learning, two fundamental concepts guide agent behavior: **policies** and **value functions**. A policy π is a mapping from states to probability distributions over actions:

(\pi: S \rightarrow \mathcal(A) \text \pi(a|s) = \Pr[A_t = a | S_t = s])

The policy represents the agent's strategy. A deterministic policy selects a single action, while a stochastic policy may select among multiple actions with specified probabilities.

The value function (v_\pi(s)) measures the expected cumulative reward from state (s) when following policy (\pi):

(v_\pi(s) = \mathbb\pi\left[\sum^{\infty} \gamma_k R_{t+k+1} \bigg| S_t = s\right])

where (\gamma \in [0,1]) is the discount factor, controlling the agent's preference for immediate versus future rewards. Similarly, the action-value function (q_\pi(s,a)) represents the expected return when taking action (a) in state (s) and thereafter following (\pi). This function becomes central to the policy improvement theorem.

The Multi-Armed Bandit Problem

The multi-armed bandit problem, dating to early probability theory (Thompson, 1933), abstracts the core exploration-exploitation dilemma. An agent faces (k) actions ("arms"), each with unknown stochastic reward distribution. At each time step (t), the agent: (1) selects action (A_t) from (k) arms, (2) receives scalar reward (R_t \sim \mathcal(\mu_a, 1)), and (3) updates action-value estimates (Q(a)). The objective is to maximize cumulative reward (\sum_^ R_t). With (k=10) arms, each reward sampled from a normal distribution with unknown mean, the agent must balance exploration (trying less-tried actions) and exploitation (selecting the best known action).

Algorithm 1: Greedy Selection

The greedy algorithm always selects the action with maximum estimated value:

(A_t = \arg\max_a Q_t(a))

With incremental updates: (Q_{t+1}(a) = Q_t(a) + \alpha[R_t - Q_t(a)]) where (\alpha) is the learning rate. Pure exploitation converges quickly to a suboptimal action, achieving only ~25% of the optimal policy's reward. The algorithm lacks exploration, fundamentally limiting performance.

Algorithm 2: Epsilon-Greedy (ε-greedy)

With probability (1-\epsilon), the agent selects greedily; with probability (\epsilon), it selects uniformly at random:

[A_t = \begin \arg\max_a Q_t(a) & \text 1-\epsilon \ \text & \text \epsilon \end]

This modification ensures continual exploration. As (T \to \infty), the regret is (\mathcal(\log T)) plus a constant term dependent on (\epsilon). Setting (\epsilon=0.1) provides good balance.

Algorithm 3: Optimistic Initial Values

Rather than initializing (Q(a) = 0) for all arms, set (Q(a) = V_{\text} >) true optimal values. Early action selections appear suboptimal after first trial, encouraging exploration without explicit randomness:

(Q_1(a) = V_{\text} \approx 5.0 \quad \text{(assuming rewards in [0,3])})

This approach is particularly effective in stationary environments and requires minimal parameter tuning.

Algorithm 4: Upper Confidence Bound (UCB)

The UCB algorithm selects actions based on both estimated value and uncertainty:

$$A_t = \arg\max_a \left[Q_t(a) + c\sqrt{\frac{\ln t}{N_t(a)}}\right]$$

where $N_t(a)$ is the number of times action a has been selected, and c is the exploration coefficient. Actions with few selections have high uncertainty, encouraging exploration; frequently selected actions are exploited if estimates are high. UCB achieves logarithmic regret bound:

$$\text{T} = \mathcal\left(\log T \sum_{a:\Delta_a > 0} \frac{\Delta_a}{\text(\mu_a || \mu^*)}\right)$$

Implementation Results and Analysis

Our implementation of all four algorithms on a 10-armed bandit problem with $\mathcal(\mu_a, 1)$ rewards reveals critical insights about exploration-exploitation tradeoffs:

Greedy (convergence at ~2.52): Converges immediately to a suboptimal arm with only 75-80% of optimal performance. The algorithm gets "stuck" early, unable to recover from poor initial estimates.

Epsilon-Greedy (convergence at ~2.80): Balanced exploration maintains long-term growth but never perfectly identifies optimal arms due to continued random exploration. As $\epsilon=0.1$ remains constant, the algorithm continues exploring suboptimal arms.

Optimistic IV (convergence at ~2.91): Natural exploration decays as estimates converge, achieving ~97% of optimal performance. Unrealistic initial values force all actions to be tried multiple times before their true values emerge.

UCB (convergence at ~2.96): Most efficient exploration strategy, rapidly identifying optimal arms while maintaining uncertainty estimates; achieves ~98.7% of theoretical optimal. The logarithmic regret bound ensures exploration focuses on promising actions.

UCB's superiority stems from principled uncertainty quantification—exploring only when justified by confidence bounds. This problem illustrates that exploration efficiency determines final performance in non-trivial RL domains, a principle that extends to all subsequent RL algorithms.

Week 2: Markov Decision Processes and Dynamic Programming Foundations

Markov Decision Processes: Formal Definition

A Markov Decision Process (MDP) is a tuple (S, A, P, R, γ) where:

- (S) is a finite set of states
- (A) is a finite set of actions
- (P(s'|s,a) = \Pr[S_{t+1} = s'|S_t = s, A_t = a]) defines stochastic transition dynamics
- (R(s,a,s') = \mathbb[R_{t+1}|S_t = s, A_t = a, S_{t+1} = s']) is the reward function
- (\gamma \in [0,1)) is the discount factor balancing immediate and future rewards

The **Markov property** specifies that the future is conditionally independent of the past given the present: (P(S_{t+1}|S_t, A_t, S_, \ldots) = P(S_{t+1}|S_t, A_t)). This assumption enables dynamic programming and reduces computational complexity from exponential to polynomial.

Bellman Equations: The Recursive Decomposition

The fundamental decomposition of value functions yields the **Bellman equations**. For any policy (\pi):

[v_\pi(s) = \sum_a \pi(a|s) \sum_{s',r} P(s',r|s,a)[r + \gamma v_\pi(s')]]

[q_\pi(s,a) = \sum_{s',r} P(s',r|s,a)[r + \gamma \sum_{a'} \pi(a'|s')q_\pi(s',a')]]

The **Bellman optimality equations** characterize optimal value functions (v_*(s) = \max_\pi v_\pi(s)):

[v_*(s) = \max_a \sum_{s',r} P(s',r|s,a)[r + \gamma v_*(s')]]

[q_*(s,a) = \sum_{s',r} P(s',r|s,a)[r + \gamma \max_{a'} q_*(s',a')]]

These equations are central to all value-based RL algorithms, providing recursive decomposition enabling efficient computation of optimal policies. The key insight is that the value of a state can be expressed in terms of immediate rewards plus the discounted values of successor states.

Environment Implementations

We implemented three increasingly complex environments to understand MDP structure:

Slippery Walk

A 1D environment where the agent navigates a path with stochastic transitions. With probability (p_{\text}), intended movements fail, causing random position jumps. Terminal states reward success and penalize failure. This environment demonstrates how policies must account for uncertainty in transitions.

Frozen Lake

A classic grid-world where the agent must reach a goal on frozen ice. Movement succeeds with probability 0.7; failure causes slipping. The environment tests robust policy learning under uncertainty. State space: (4 \times 4 = 16) states; action space: 4 directions. Optimal policies exhibit caution around dangerous areas while moving efficiently toward the goal.

Lights Out (4×4)

A custom MDP for the Tiger Electronics puzzle. The state space is (2^{16} = 65,536) (each cell is on/off), actions correspond to 16 buttons (toggle plus adjacent lights), and only the all-lights-off state is terminal with reward +1. This demonstrates how RL scales beyond toy examples to moderately complex combinatorial spaces requiring learned structure and intelligent search.

Bellman Backup Operations

The practical implementation of Bellman equations requires iterative updates. For state evaluation:

[v_{k+1}(s) \leftarrow \sum_a \pi(a|s) \sum_{s',r} P(s',r|s,a)[r + \gamma v_k(s')]]

For optimal value backup:

[v_{k+1}(s) \leftarrow \max_a \sum_{s',r} P(s',r|s,a)[r + \gamma v_k(s')]]

These are called **Bellman backups** or **bootstrapping**, expressing value functions in terms of themselves at other states, enabling iterative refinement until convergence to the true value functions.

Week 3: Policy Iteration and Value Iteration Algorithms

Dynamic Programming Methods Overview

Two fundamental algorithms emerge from Bellman equations for computing optimal policies in known MDPs:

Policy Iteration alternates between: (1) **Policy Evaluation**: compute (v_\pi(s)) for current policy (\pi) by solving the linear system (v_\pi(s) = \sum_a \pi(a|s) \sum_{s',r} P(s',r|s,a)[r + \gamma v_\pi(s')]), and (2) **Policy Improvement**: update policy greedily (\pi'(s) = \arg\max_a q_\pi(s,a)).

Value Iteration directly applies the Bellman optimality equation:

[v_{k+1}(s) \leftarrow \max_a \sum_{s',r} P(s',r|s,a)[r + \gamma v_k(s')]]

Both converge to (v_*) *and* (\pi_*). Policy iteration converges faster (fewer outer iterations) but requires solving linear systems per iteration. Value iteration requires more iterations but simpler computation per step.

Problem 1: Jack's Car Rental

Problem Statement: Jack manages two rental locations. Daily customer requests ((\text(\lambda))) rent available cars for 10 each. Returns follow Poisson distributions. Jack can move cars overnight at 2 per car (maximum 5). Locations hold maximum 20 cars.

MDP Formulation:

- State: (s = (n_1, n_2)) where (n_i) is cars at location (i), (n_i \in {0,\ldots,20})
- Action: (a \in {-5,\ldots,5}) (net cars moved from location 1 to 2)
- Transitions: Poisson distributions with (\lambda_{\text} = (3,4)) and (\lambda_{\text} = (3,2))

Reward Structure: [R(s,a,s') = 10(r_1 + r_2) - 2|a|]

Results: Policy iteration converged in 4 iterations with 10 policy evaluation sweeps each. The optimal policy exhibits clear structure: transfer cars from location 2 to location 1 when location 1 is empty, maintain balanced inventory of 10-15 cars per location. Optimal expected daily reward: 37.50 (vs 28 for random). This demonstrates how DP methods discover non-trivial optimal strategies in stochastic environments.

Problem 2: Gambler's Problem

Problem Statement: Gambler bets on coin flip sequences, wagering integer dollars. Heads: double stake. Tails: lose stake. Goal: reach 100. Decide optimal wager at each wealth level.

MDP Formulation:

- State: (s \in {0,1,\ldots,100}) (capital)
- Action: (a \in {0,\ldots,\min(s, 100-s)}) (amount wagered)
- Transition: (P(s+a|s,a) = 0.5) (win), (P(s-a|s,a) = 0.5) (lose), with terminals at 0 and 100

Value Semantics: (v_*(s) = \Pr[\text 100 | \text s])

Bellman Optimality:

[v_*(s) = \max_{a \in {0,\ldots,\min(s,100-s)}} [0.5 \cdot v_(s+a) + 0.5 \cdot v_*(s-a)]]

Implementation: Value iteration with convergence threshold (\theta = 10^{-6}) required ~50 iterations. Optimal policy exhibits elegant structure: at (s=50) (fair coin), (a^=1) (minimal betting); at (s < 25), (a^=s) (all-or-nothing); at (s > 75), (a^*=100-s) (one shot to victory). This demonstrates non-smooth optimal policies with discrete strategy jumps, showing that optimal solutions can exhibit counterintuitive structure.

Lights Out with Value Iteration

Applying value iteration to 4×4 Lights Out: convergence required ~300 iterations with (\gamma=0.99). State space size (2^{16}) is manageable but near tabular method limits. Optimal policy solved Lights Out in 9-11 moves (near-optimal). Value function showed clear gradient from unsolved states (low value) to goal ((v_*=1)). The environment demonstrates that combinatorial grid problems benefit from problem-specific structure; naive solvers use BFS; RL learns this implicitly through value functions.

Week 4: Deep Reinforcement Learning and the 15-Puzzle

Problem Motivation and Complexity

The 15-puzzle, invented in 1874, presents a 4×4 grid with 15 numbered tiles and one empty space. Tiles adjacent to the space slide. Goal: numerical ordering (1-15 left-to-right, top-to-bottom).

Complexity: State space (\frac{16!}{2} = 10^{13}) states (half unreachable due to parity); action space 2-4 per state. Tabular methods are impossible—(10^{13} \times 4) transitions exceed memory and computation. Unlike previous problems, the 15-puzzle demands decomposition and learned representations rather than tabular value functions.

Approach: Subgoal Decomposition with Hierarchical Learning

Following Amshali (2017), we decomposed into three learned subproblems:

Phase 1: Solve First Row (tiles 1-4 correct positions, state space (\approx $10^{8})$)

Phase 2: Solve Second Row (tiles 5-8 while preserving row 1, state space (\approx $10_{8})$)

Phase 3: Solve Remaining Tiles (tiles 9-15 while preserving rows 1-2, state space (\approx 10^{11}))

Hierarchical decomposition reduces effective state space per phase and mirrors human problem-solving strategies.

Implementation Details

State representation: 16-dimensional integer vector (tiles 0-15, where 0 is empty)

Feature extraction: Since 15-puzzle requires non-linear approximation:

- Distance metric: (\phi_1(s) = \sum_^{15} $|x_i - x_i|$ + $|y_i - y_i|$) (Manhattan distance)
- Solvability: (\phi_2(s) =) permutation parity
- Learned features: CNN with 2 layers (16→32→16 filters)

Algorithm: Hybrid breadth-first search with value-guided rollout:

1. For states within (d=6) moves of goal: BFS guarantees solution
2. Distant states: learned value function selects moves
3. Online refinement: Update estimates as solutions found

Results and Analysis

Performance:

- Successfully solved 15-puzzle from random states
- Solution length: 60-70 moves (optimal typically 50-55)
- Computation: ~2-5 minutes on CPU
- Learning curve: Exponential improvement over 500 episodes

Success factors: (1) Subgoal decomposition reduced per-phase complexity, (2) BFS guarantee ensured solutions, (3) Value guidance provided anytime solutions, (4) Hybrid method combined search with learned components.

Limitations: Solution suboptimal vs IDA* (~53 moves), requires domain knowledge (decomposition structure), not end-to-end learned.

Theoretical Connection: Options Framework

Subgoal decomposition maps to the **options framework** (Barto et al., 1999), extending MDPs with temporally extended actions:

[\mathcal = {\langle I_o, \pi_o, \beta_o \rangle}]

Each option specifies: (1) **Initiation set** (I_o) (states where available), (2) **Policy** (\pi_o) (behavior while executing), (3) **Termination** (\beta_o(s')) (probability of termination). Our puzzle phases constitute options: "solve-row-1" (policy specialized for row 1, terminates when row 1 solved), "solve-row-2" (available after row 1), "solve-remaining" (after rows 1-2). Hierarchical structure with options represents a key direction for scaling RL to complex domains.

Theoretical Synthesis and Convergence Guarantees

Convergence of Value Iteration

Theorem (Convergence of Value Iteration): For finite MDPs, value iteration converges to (v_*) with exponential rate:

[|v_k - v_|*infty \leq \gamma^k |v_0 - v*|_\infty]

Since (\gamma < 1), convergence is guaranteed in finite time. Practically, we stop when (\max_s |v_{k+1}(s) - v_k(s)| < \theta) (convergence threshold).

Policy Improvement Theorem

Theorem: If (\pi') is greedy with respect to (v_\pi), then (v_{\pi'}(s) \geq v_\pi(s)) for all states (s). Proof: For any state (s):

[v_\pi(s) = \sum_a \pi(a|s) q_\pi(s,a) \leq \sum_a \pi'(a|s) q_\pi(s,a)]

By definition of (\pi') greedy:

[\sum_a \pi'(a|s) q_\pi(s,a) = q_\pi(s,\pi'(s)) \leq v_{\pi'}(s)]

Therefore (v_\pi(s) \leq v_{\pi'}(s)). This theorem guarantees policy iteration monotonically improves.

Regret Bounds for Bandits

For (\epsilon)-greedy with bandit rewards sampled i.i.d. from unknown distribution, cumulative regret satisfies:

[\text(T) = (k-1)T \epsilon \cdot \mathbb[R_{\text}] + (1-\epsilon)T \cdot \mathbb[\text]]

The first term comes from exploration (trying suboptimal arms with probability (\epsilon)), the second from not always selecting the best known arm. UCB algorithms achieve tighter regret bounds through information-theoretic analysis.

Conclusion and Future Directions

Over four weeks, we progressed from bandit algorithms through dynamic programming to hybrid deep RL for the 15-puzzle. Key insights:

1. **Exploration matters**: UCB and Optimistic IV dramatically outperform greedy approaches through principled uncertainty management
2. **Bellman equations are universal**: Recursive value decomposition enables RL across diverse domains—bandits, car rentals, puzzles
3. **Scalability requires structure**: Tabular methods fail at (10^{13}) states; learned representations and hierarchical decomposition become essential
4. **Theory-practice gaps**: Optimal algorithms require approximations and domain knowledge for practical implementation

---